Joana Amézaga Velasco

RESPUESTAS A LAS PREGUNTAS DEL CHECKPOINT 3

1. **¿Cuáles son los tipos de Datos en Python?**

Los tipos de datos son numéricos, strings, booleanos, byte y matrices de bytes, none, y catergoría de estructura de datos: listas, tuples, sets y diccionarios.

- Booleanos: verdadero o falso.
- Numéricos: dentro de los cuales tenemos los enteros, los de punto flotante, los decimales, los complejos. Python detecta el tipo de número que estamos usando, no hace falta que se lo indiquemos.
- Strings: cadenas de texto. Para identificarlo entrecomillaremos el texto, se pueden usar comillas simples o dobles pero tenemos que tener cuidado si en la propia cadena también las hay, por ejemplo "it's a dog". Para evitar problemas combinamos uno y otro tipo o "anulamos" el efecto de la comilla con una barra \ antes de ella.
- Bytes and byte arrays: bytes y matrices de bytes. Esto es avanzado y en principio no lo usaremos aquí.
- None: ninguno. Es como null. Sirve para crear una variable, pero a la que no queremos asignar un valor todavía.
- Categoría de estructura de datos. Hay cuatro maneras en las que podemos estructurar las colecciones. Todo esto es avanzado y lo veremos más adelante.
    - Lists: parecido a una matriz.
    - Tuples: similares a las listas pero tienen características únicas.
    - Sets (conjuntos): similares a las listas pero tienen características únicas.
    - Dictionaries: te dan la capacidad de tener valores pares.

2. **¿Qué tipo de convención de nomenclatura deberíamos utilizar para las variables en Python?**

Se escriben en minúsculas y no debemos usar L minúscula ni i mayúscula porque se confunden entre sí. Tampoco o mayúscula porque se confunde con cero.

Para separar las palabras se usa guion bajo y debemos intentar que los nombres sean descriptivos, para no tener que usar comentarios.

3. **¿Qué es un Heredoc en Python?**

Heredoc es el texto en múltiples líneas en programación. Si tengo varias strings y las quiero convertir en un Heredoc, lo hago con 3 comillas antes y después de todo el texto, pueden ser simples o dobles.

Normalmente las comillas se ponen una línea antes y una línea después del texto, para que el código se lea mejor. Esto hace que al imprimirlo aparezcan líneas en blanco. Para evitar esto podemos poner .strip() justo después de las segundas comillas y esto elimina las líneas en blanco.

```
1 # heredoc
2
3 contenido = """
4 Nullam id dolor id nibh ultricies vehicula ut id elit. Nullam quis risus eget
5
6 Vestibulum id ligula porta felis euismod semper. Cum sociis natoque penatibus
7
8 Integer posuere erat a ante venenatis dapibus posuere velit aliquet.
9 """.strip()
10
11 print(contenido)
```

Para reescribir un heredoc en una sola línea, lo que nos puede hacer falta en algunos momentos, usaremos repr cuando queramos imprimirlo. En la línea del resultado veremos \n donde haya líneas en blanco.

```
1 # heredoc
2
3 contenido = """
4 Nullam id dolor id nibh ultricies vehicula ut id elit. Nullam quis r
5
6 Vestibulum id ligula porta felis euismod semper. Cum sociis natoque
7
8 Integer posuere erat a ante venenatis dapibus posuere velit aliquet.
9 """
10
11 print(repr(contenido))
```

Entrada del programa

Salida

```
'\nNullam id dolor id nibh ultricies vehicula u

[Execution complete with exit code 0]
```

4. ¿Qué es una interpolación de cadenas?

Es una acción en la que de introducimos variables u otras cadenas dentro de cadenas. Por ejemplo, en un correo electrónico que escribimos una sola vez pero queremos que se vea personalizado, en el lugar del nombre incrustamos la variable nombre, y lo mismo con el resto de datos. En un momento dado también podríamos incrustar una nueva cadena.

Actualmente se hace de la siguiente manera: La **f** antes del string significa: "Sustituye lo que hay entre llaves {} por variables o expresiones". Técnicamente, dentro de los corchetes puedo poner cualquier código Python.

```
1 nombre = "Marisa"
2 saludo = f"Hola {2+2}"
3 print(saludo)
4
```

Entrada del

Salida

```
Hola 4
```

Si nos encontramos con alguna aplicación programada de la manera antigua será usando el indicador numérico en base 0 como se ve en la siguiente imagen:

```python
nombre = "Mirari"
edad = 105
producto = "lechuga"

saludo = "Hola {0}, tienes {1} años y has comprado {2}".format(nombre, edad, producto)

print(saludo)
```

5. **¿Cuándo deberíamos usar comentarios en Python?**

Se deben evitar porque quedan obsoletos cuando hacemos cambios, ya que los comentarios no solemos actualizarlos. Lo mejor es que los nombres que asignemos a las variables sean autoexplicativas para que no haga falta comentar.

6. **¿Cuáles son las diferencias entre aplicaciones monolíticas y de microservicios?**

Aplicación monolítica: la función de todo el sistema se basa en una aplicación, es más rápida de desarrollar. Se hace la base y se puede ampliar. Pueden ser más rápidas que las de microservicios porque no tienen que comunicarse a través de las API. Todo está en el mismo sistema, en el mismo servidor, en la misma máquina, y todos usan el mismo sistema de archivos. Mantener una aplicación monolítica es un desafío, porque un pequeño cambio puede tener un efecto dominó y causar varios problemas.

Aplicación de microservicios: cada característica es su propia aplicación. Cada una reside en su propio servidor, y se conectan entre sí, normalmente usando aplicaciones basadas en API JSON. Si creas bien un microservicio, los demás elementos no deberían depender de él, al contrario de lo que pasa con la monolítica. Así se pueden desarrollar y probar individualmente. Tenemos una gran capacidad de ampliar. Suele ser apropiado en aplicaciones grandes, no tanto en pequeñas porque se tarda mucho más en desarrollar y es más cara para los clientes. Es habitual que la aplicación la desarrollen diferentes desarrolladores ubicados en distintos lugares. Lo que hace uno no interfiere en lo que hace otro, y luego se conectan. Es difícil para desarrolladores novatos.